

Autoregressive Transformer Language Models: From Next-Token Prediction to Text Generation

A technical report based on the supplied NLP textbook excerpt and primary research literature

Abstract

This report explains how modern autoregressive language models generate text, starting from the next-token prediction objective illustrated in the supplied textbook excerpt. The central idea is simple: a language model estimates a probability distribution over the next token given the tokens that precede it, and text generation repeatedly samples or selects from that distribution. The report then connects this objective to the Transformer architecture introduced by Vaswani et al. (2017), the GPT line of decoder-style language models, scaling behavior, and instruction-following. Particular attention is given to causal self-attention, tokenization, logits and softmax, autoregressive decoding, temperature and sampling, and the distinction between pretraining and post-training. The goal is not to present a new research contribution, but to provide a technically accurate undergraduate-level document that can serve as a study note, portfolio document, or foundation for a future research project.

Publication note: This is a technical report/tutorial, not an original research paper. It synthesizes established research and the supplied textbook excerpt. A research paper would normally require a novel question, experiment, dataset, theoretical contribution, or other original result.

1. Introduction: What Is a Language Model?

A language model (LM) assigns probabilities to sequences of linguistic units. At the most basic level, it answers a question such as: “Given the text I have already seen, what is likely to come next?” The supplied textbook excerpt presents exactly this intuition. A large language model receives a context such as “So long and thanks for” and produces a probability distribution over possible next tokens. The distribution might assign relatively high probability to “all,” while assigning smaller probabilities to alternatives such as “the,” “your,” or “that.” The important point is that the model does not directly output a sentence as a single indivisible object; it predicts the next unit and can then use that prediction as part of the context for the next prediction.

Formally, for a token sequence $x_1, x_2, \dots, x_T$, an autoregressive language model factorizes the probability of the complete sequence as: $P(x_1, x_2, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, \dots, x_{t-1})$. This factorization is fundamental. Instead of learning a probability for every possible complete sentence, the model learns a sequence of conditional next-token distributions. During training, the previous tokens are known from the dataset, so the model can be optimized to predict the observed next token. During generation, however, the model must feed its own newly generated tokens back into the context. This is the mechanism described by the textbook’s second figure: prediction becomes generation through repeated sampling from the model’s output distribution.

The modern large-language-model lineage combines this language-modeling objective with neural architectures capable of processing long contexts. GPT-2 described a large unsupervised language model trained to predict the next token over Internet text, while GPT-3 demonstrated that scaling an autoregressive language model can produce strong few-shot performance without task-specific gradient updates at inference time [3,4].

2. From Tokens to a Probability Distribution

Before a Transformer can process text, the text must be represented as a sequence of discrete tokens. A token is not necessarily a complete word. Modern language models commonly use subword-level tokenization so that common words can be represented efficiently while rare or previously unseen words can be decomposed into smaller units. Sennrich, Haddow, and Birch showed that subword units provide a practical solution to the open-vocabulary problem in neural machine translation [5]. GPT-2, for example, used a vocabulary of 50,257 tokens and operated over sequences of these discrete units [6].

Each token is mapped to an integer ID and then to a dense vector called an embedding. If the vocabulary contains V tokens and the model's hidden dimension is d , the embedding table can be viewed as a matrix $E \in \mathbb{R}^{(V \times d)}$. Looking up token i selects row i of E . The resulting vectors are combined with positional information so that the model can distinguish different positions in the sequence. The original Transformer used positional encodings because, unlike recurrent networks, self-attention alone does not inherently encode the order of tokens [1].

The Transformer produces a hidden representation for each position. At the final prediction stage, the representation at the current position is transformed into a vector of V real-valued scores called logits. A softmax function converts these scores into a probability distribution: $P(\text{token}=i \mid \text{context}) = \exp(z_i) / \sum_j \exp(z_j)$. The highest-probability token can be selected by greedy decoding, but generation does not have to be deterministic. Sampling allows the model to choose among multiple plausible tokens. This distinction is important because the model's output is fundamentally a distribution, not simply a single "correct answer."

3. The Transformer and Causal Self-Attention

The Transformer was introduced by Vaswani et al. in 2017 as an architecture based entirely on attention mechanisms, removing recurrence and convolution from the core sequence-transduction architecture [1]. Its key operation is self-attention. Given query, key, and value matrices Q , K , and V , scaled dot-product attention is: $\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k})V$. Intuitively, each position can calculate how strongly it should attend to other positions. The dot product between a query and a key measures compatibility; the softmax turns those scores into weights; and the weighted sum of value vectors produces the attended representation.

For autoregressive generation, attention must be causal. A token at position t must not be allowed to use information from future positions $t+1$, $t+2$, and so on, because those tokens would not be available when the model is actually generating text. A causal mask therefore prevents attention from flowing from a position to future positions. During training, the model can process many positions in parallel while still respecting this causal constraint. This is one of the reasons the Transformer is computationally attractive compared with strictly sequential recurrent architectures [1].

The original Transformer was an encoder-decoder architecture, but GPT-style language models use a decoder-only form. In a decoder-only model, the stack repeatedly transforms the token representations using masked self-attention and feed-forward sublayers. Residual connections and normalization help optimization and information flow. GPT-2 modified the Transformer block configuration and normalization strategy while scaling the architecture substantially; its largest model in the original report had more than an order of magnitude more parameters than the original GPT [6].

4. Autoregressive Text Generation: Turning Prediction into Generation

Suppose the prompt is “The capital of France is”. The model computes a probability distribution over the next token. Imagine, for illustration, that “Paris” receives a high probability. If the decoder selects “Paris,” the context becomes “The capital of France is Paris.” The model then performs another forward pass for the next token. This loop continues until a stopping condition is reached, such as an end-of-sequence token or a maximum generation length. This is the core algorithmic idea shown in the supplied textbook figure.

The decoding strategy determines how the probability distribution is converted into a token. Greedy decoding always selects $\text{argmax}_i P(x=i \mid \text{context})$. It is simple and deterministic, but it can produce repetitive or locally optimal text. Sampling instead draws a token according to the distribution. Temperature modifies the sharpness of the distribution by dividing logits by a temperature parameter τ before softmax. Lower temperatures make the distribution more concentrated; higher temperatures make it flatter. Other decoding procedures can restrict the candidate set, such as selecting from a limited group of high-probability tokens.

A critical distinction is therefore between the model and the decoding algorithm. The neural network computes the distribution; decoding decides how to use that distribution to construct a sequence. Two generations from the same model can differ because the decoding process may sample different tokens. Conversely, deterministic decoding can reproduce the same output for the same input and configuration. The textbook’s phrase “turning a predictive model into a generative model” is therefore best understood as repeatedly applying the predictive conditional distribution to an expanding context.

Training uses the same mathematical objective from the opposite direction. Given a corpus of token sequences, the model is trained to maximize the likelihood of observed next tokens, equivalently minimizing negative log-likelihood or cross-entropy loss. Because the training tokens are known, the model does not have to wait for its own predictions to proceed through the sequence. This creates a major difference between training and inference: training can evaluate many next-token predictions in parallel, while generation is inherently sequential at the token level.

5. Scaling, Instruction Following, and Limitations

Once the autoregressive objective and Transformer architecture were established, researchers found that increasing model size, dataset size, and compute could systematically improve language-model loss. Kaplan et al. reported empirical power-law relationships between language-model performance and model size, dataset size, and compute over broad ranges [7]. GPT-3 then demonstrated the practical consequences of scaling: the model contained 175 billion parameters and showed substantially improved few-shot performance without updating its weights for each task [4]. These results helped establish scaling as a central engineering strategy for large language models.

However, next-token prediction alone does not guarantee that a model will follow user instructions, provide truthful information, or behave safely. Ouyang et al. showed that post-training with human feedback can substantially improve instruction following. Their InstructGPT pipeline combined supervised fine-tuning on human demonstrations with a reward model trained from human rankings and reinforcement learning from human feedback (RLHF) [8]. This illustrates an important conceptual separation: pretraining teaches a model broad language and world-pattern representations, while post-training can shape how the model behaves when interacting with users.

Autoregressive language models also have important limitations. A model can generate fluent text that is factually incorrect, because its training objective is to predict likely continuations rather than to guarantee truth. Large web corpora can contain biases, inaccuracies, and duplicated or contaminated material. OpenAI's GPT-2 documentation explicitly warned that the training data contained biases and factual inaccuracies and recommended careful evaluation of generated text [9]. In addition, model quality depends on tokenization, data quality, architecture, optimization, inference strategy, and evaluation methodology. These limitations matter when a language model is used in applications where correctness is more important than linguistic fluency.

From an engineering perspective, the complete system can therefore be viewed as a pipeline: raw text → tokenization → token embeddings and positional information → Transformer blocks with causal self-attention → logits → probability distribution → decoding → generated tokens → updated context → repetition of the decoding loop. The mathematical simplicity of next-token prediction hides substantial systems complexity, but the core generative mechanism remains remarkably direct.

6. Conclusion

The central concept from the supplied textbook excerpt is the foundation of modern autoregressive language generation: predict the next token from the current context, then append the selected token and repeat. Transformers make this process powerful by using self-attention to build context-dependent representations, while causal masking preserves the left-to-right constraint required for generation [1]. GPT-style models demonstrate how a decoder-only Transformer can be trained at scale with a next-token objective [3,4,6]. Scaling research and instruction-following work further show that model size, data, compute, and post-training all influence practical capabilities [7,8].

For an undergraduate computer-science student, the most useful mental model is not “an LLM writes text like a human.” A better model is: “an LLM is a neural function that maps a token context to a probability distribution over the next token; generation repeatedly applies that function while a decoding algorithm chooses the next token.” Understanding that loop provides a foundation for studying attention, Transformer architectures, training objectives, inference optimization, fine-tuning, alignment, and eventually modern AI systems research.

References

- [1] A. Vaswani et al., “Attention Is All You Need,” arXiv:1706.03762, 2017.
- [2] A. Radford et al., “Improving Language Understanding by Generative Pre-Training,” OpenAI, 2018.
- [3] A. Radford et al., “Language Models are Unsupervised Multitask Learners,” OpenAI technical report, 2019.
- [4] T. B. Brown et al., “Language Models are Few-Shot Learners,” NeurIPS 2020.
- [5] R. Sennrich, B. Haddow, A. Birch, “Neural Machine Translation of Rare Words with Subword Units,” ACL 2016.
- [6] A. Radford et al., “Language Models are Unsupervised Multitask Learners,” GPT-2 architecture and training details, 2019.
- [7] J. Kaplan et al., “Scaling Laws for Neural Language Models,” arXiv:2001.08361, 2020.
- [8] L. Ouyang et al., “Training Language Models to Follow Instructions with Human Feedback,” NeurIPS 2022.
- [9] OpenAI, “GPT-2 Repository and Model Documentation,” 2019.